BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CUỐI KỲ

IOT-BASED MEDICAL ALERT SYSTEM FOR ELDERLY PEOPLE

Môn: Vạn vật kết nối

Giảng viên hướng dẫn: Ths.Đinh Công Đoan

Lớp: INOT231780_02

Nhóm: 7

Thành viên: Phạm Trần Thiên Đăng - 23110203

Lê Vũ Hải - 23110209

Huỳnh Ngọc Thắng - 23110327

Thành phố Hồ Chí Minh, tháng 5 năm 2025

BẢNG PHÂN CÔNG NHIỆM VỤ

Họ và tên	Mã số sinh viên	Nhiệm vụ	Tỷ lệ hoàn thành
Huỳnh Ngọc Thắng	23110327	ESP32, Blynk, GPS, Buzzer, Lắp mạch, Hỗ trợ báo cáo.	100%
Phạm Trần Thiên Đăng	23110203	OLED, Button, MPU5060, PowerPoint, Hàn mạch	100%
Lê Vũ Hải	23110209	MAX30102, Battery, Flask Server (Web), Bài báo cáo.	100%

Lời nhận xét:

……
……
……
……
……
……
……
……
……

Điểm	Chữ ký

LỜI CẢM ƠN

Nhóm xin bày tỏ lòng biết ơn chân thành đến Ths. Đinh Công Đoan – người đã trực tiếp giảng dạy, hướng dẫn và luôn tận tâm hỗ trợ em trong suốt quá trình học tập và thực hiện đồ án *"IoT-based Medical Alert System For Elderly People"*. Nhờ có sự chỉ dẫn tận tình và sự khuyến khích, tạo điều kiện của Thầy, em không chỉ tiếp thu được những kiến thức quý giá về chuyên môn mà còn học được cách tư duy logic, tiếp cận vấn đề một cách khoa học và thực tế hơn. Đồ án lần này không chỉ là cơ hội để em áp dụng lý thuyết vào thực tiễn, mà còn là dịp để em hiểu sâu sắc hơn về tầm quan trọng của công nghệ trong việc hỗ trợ và nâng cao chất lượng cuộc sống, đặc biệt là đối với người cao tuổi. Em xin trân trọng cảm ơn Thầy vì đã đồng hành, định hướng và luôn sẵn lòng chia sẻ kiến thức, kinh nghiệm để giúp em hoàn thành đề tài một cách hiệu quả nhất. Kính chúc Thầy dồi dào sức khỏe, luôn tràn đầy nhiệt huyết và gặt hái nhiều thành công hơn nữa trong sự nghiệp trồng người cao quý.

MỤC LỤC

MỤC LỤC HÌNH ẢNH .....

PHẦN 1: MỞ ĐẦU 1

1. Tóm tắt ý tưởng ứng dụng báo cáo 1

2. Đặt vấn đề 1

2.1 Tóm lược những nghiên cứu trong và ngoài nước liên quan đến đề tài 1

2.2 Tính cấp thiết cần nghiên cứu của đề tài 2

2.3 Một số tài liệu có liên quan 2

2.4 Lý do chọn đề tài 2

2.5 Mục tiêu đề tài 3

2.6 Đối tượng và phạm vi nghiên cứu 3

2.7 Phương pháp nghiên cứu 3

2.8 Bố cục đề tài 3

PHẦN 2: NỘI DUNG 4

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI 4

1.1 Tổng quan IoT trong y tế 4

1.2 Tổng quan về hệ thống phát hiện khẩn cấp 4

1.3 Giới thiệu về nền tảng Blynk hỗ trợ khía cạnh đề tài. 4

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU 5

2.1 Giới thiệu yêu cầu 5

2.2 Sơ đồ nguyên lý 5

2.3 Nguyên tắc làm việc 6

2.4 Lựa chọn giải pháp 12

CHƯƠNG 3: NHỮNG KIẾN THỨC LIÊN QUAN 15

3.1 Kiến thức liên quan phần cứng (chức năng cơ bản và thông số kĩ thuật) 15

3.2 Kiến thức liên quan phần mềm (kiến thức áp dụng và tư duy lập trình)..... 17

CHƯƠNG 4: ỨNG DỤNG 23

4.1 Sơ đồ khối ứng dụng..... 23

4.2 Quy trình xây dựng..... 24

4.3 Những thành tựu đạt được, khó khăn và giải pháp..... 28

PHẦN 3: KẾT LUẬN..... 33

TÀI LIỆU THAM KHẢO .....

MỤC LỤC HÌNH ẢNH

Hình 2.1: Sơ đồ khối nguyên lý 6
Hình 2.2: Mô phỏng trigger3 xảy ra 7
Hình 2.3: Code mô phỏng thông báo 8
Hình 2.5: Code tính toán nhịp tim 9
Hình 2.6: Code tính toán SpO2 10
Hình 2.7: Code thực thi GPS 11
Hình 2.8: Code kiểm tra Blynk 12
Hình 2.9: ESP32 12
Hình 2.10: MAX30102 13
Hình 2.11: MPU6050 13
Hình 2.12: Neo-6M V2 13
Hình 2.13: OLED SSD1306 14
Hình 2.14: Battery 18650 14
Hình 3.1: Code quản lý thời gian thực 18
Hình 3.2: Code xử lý dữ liệu GPS 19
Hình 3.3: Code cập nhật thời gian 20
Hình 3.4: Cập nhật màn hình OLED 21
Hình 3.5: Code xử lý nút bấm 22
Hình 4.1: Sơ đồ khối ứng dụng 23
Hình 4.2: Mô hình ứng dụng thô cho bước 1 24
Hình 4.3: Mô hình mô phỏng cho bước 2 24
Hình 4.4: Module ESP32 cho bước 3 25
Hình 4.5: Thiết kế hoàn thiện cho bước 4 26
Hình 4.6: Sản phẩm mô phỏng bước 5 27
Hình 4.7: Phân tích cấu hình Battery 28
Hình 4.8: Hệ thống giám sát bằng Web 28
Hình 4.9: Hệ thống giám sát bằng Blynk 29
Hình 4.10: Cảnh báo trên OLED 30
Hình 4.11: Định vị GPS 31
Hình 4.12: Cấu hình màn OLED 32

PHẦN 1: MỞ ĐẦU

1. Tóm tắt ý tưởng ứng dụng báo cáo

Tổng quan nhất về đồ án với chủ đề **"Iot-based Medical Alert System For Elderly People"**, xây dựng hệ thống giám sát sức khỏe và phát hiện khẩn cấp cho người cao tuổi dựa vào công nghệ IoT.

Phần thứ nhất chủ yếu trình bày về những cơ sở hình thành và xây dựng nên chủ đề của đồ án (phần mở đầu).

Phần thứ hai sẽ trình bày lần lượt cơ sở lý thuyết dựa vào phân tích chủ đề, từ đó áp dụng thực tiễn cụ thể đồ án để đánh giá và sử dụng hiệu quả nhất (phần nội dung).

Phần thứ ba chủ yếu đánh giá lại quá trình, hiệu suất và thực tiễn vấn đề (phần kết luận)

2. Đặt vấn đề

2.1 Tóm lược những nghiên cứu trong và ngoài nước liên quan đến đề tài

Quốc tế đã có những nghiên cứu nổi bật liên quan đề tài:

+ *IoT-based Fall Detection System for Elderly Healthcare (2018)* được nghiên cứu với S.M.Riazul Islam và các cộng sự. Phát hiện ngã và giám sát sức khỏe.

+ *Dự án nghiên cứu của WolkAbout (2019),* Abuelómetro. Phục vụ tại viện dưỡng lão dùng để giám sát nhịp tim, SpO2 và nhiệt độ.

+ *City4Age (2020)* đây là dự án của H2020 City4Age, liên minh các nhà nghiên cứu châu Âu. Phát hiện sự giảm nhận thức nhẹ và tình trạng yếu đuối.

Trong nước đã có thành tựu đáng tự hào liên quan đến đề tài:

+ *Hệ thống giám sát sức khỏe IoT cho bệnh nhân cao tuổi (2019)* được thực hiện bởi nhóm nghiên cứu tại Đại học Bách Khoa TP.HCM. Sử dụng Arduino, giám sát nhịp tim và nhiệt độ.

+ *Hộp thuốc thông minh dựa trên IoT (2020)* được nghiên cứu bởi nhóm nghiên cứu tại Đại học Công nghệ, Đại Học Quốc Gia Hà Nội. Nhắc nhở uống thuốc qua ứng dụng di động và tin nhắn SMS.

+ *Hệ thống cảnh báo sức khỏe sức khỏe từ xa cho người cao tuổi (2021)* được nghiên cứu bởi thành viên của nhóm nghiên cứu của Đại học Sư phạm Kỹ Thuật TP.HCM. Được sử dụng trong việc giám sát nhịp tim, SpO2 và gửi cảnh báo SMS.

2.2 Tính cấp thiết cần nghiên cứu của đề tài

Người lớn tuổi hiện nay đang phải đối mặt với rất nhiều vấn đề về sức khỏe nghiêm trọng do nhiều nguyên nhân từ cuộc sống, tổ chức WHO ghi nhận nhiều nguyên nhân hàng đầu như ngã, nhịp tim bất thường và tình trạng thiếu oxy trong máu. Tại Việt Nam, tốc độ già hóa dân số ngày càng tăng, đề tài này sẽ trở thành đối tượng cần được nghiên cứu và tìm ra giải pháp ngay lập tức. Để đáp ứng nhu cầu, nhóm 7 đã thực hiện nghiên cứu và xây dựng giải pháp, hỗ trợ kịp thời và giảm rủi ro mà còn mang lại an tâm người cao tuổi và gia đình. Giúp góp phần nâng cao chất lượng cuộc sống trong bối cảnh hiện đại ngày nay.

2.3 Một số tài liệu có liên quan

Giáo trình Cảm biến Quyển 1 + 2 , tác giả Phan Quốc Phô, nhà xuất bản Bách Khoa Hà Nội.

Mô-đun gia tốc kế MPU6050 và con quay hồi chuyển (2021), Điện dân dụng.

Lập trình STM32 với Oled LCD SSD1306 (2021), Nguyễn Khuê.

Tìm hiểu về Buzzer, Điện tử tương lai.

GPS NEO giao tiếp Arduino, Định vị GPS Neo 6M + LCD1602 + Arduino (2022), Điện tử Nhật Tùng.

2.4 Lý do chọn đề tài

Đề tài **"Iot-based Medical Alert System For Elderly People"** được lựa chọn do sự xem xét cách toàn diện của nhóm, khai thác hai khía cạnh là thực tiễn và khả thi. Đầu tiên, đề tài đã đáp ứng đầy đủ nhu cầu thiết yếu đời sống là giám sát sức khỏe của cá nhân ở độ tuổi cao. Do đây chỉ nghiên cứu mức học tập, khả thi về kiến thức và chi phí có thể đáp ứng được, vi điều khiển ESP32 và nền tảng Blynk công nghệ giá cả bình dân. Sáng tạo đây là tích hợp nhiều vấn đề nghiên cứu trong thiết bị nhỏ gọn phát hiện và giám sát người lớn tuổi một cách thông minh và tiện lợi thông qua nhiều nền tảng khác nhau. Lý do cuối

cùng, niềm đam mê nhóm về lĩnh vực công nghệ, đặc biệt trong lĩnh vực IoT để hỗ trợ cộng đồng người cao tuổi một cách hiệu quả.

2.5 Mục tiêu đề tài

Đồ án với chủ đề **"Iot-based Medical Alert System For Elderly People"** được thực hiện bởi thành viên của nhóm 7, đồ án xây dựng và thực hiện một nền tảng, hệ thống trong việc giám sát và phát hiện khẩn cấp những vấn đề xảy ra đối với người lớn tuổi trong tình huống bất ngờ của đời sống. Gửi cảnh báo đến người thân và nhân viên y tế trong tình huống người lớn tuổi gặp phải thông qua nền tảng IoT. Đối với nền tảng lập trình, mục tiêu là tích hợp cảm biến thuận tiện cho việc sử dụng, phát triển thuật toán phát hiện ngã chính xác, đo nhịp tim lẫn SpO2 và hiển thị lên OLED. Gửi cảnh báo đến nền tảng Blynk, email, xDroid và Web của nhóm nghiên cứu.

2.6 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Hệ thống IoT giám sát sức khỏe và phát hiện khẩn cấp.

Phạm vi nghiên cứu: Không gian là khu vực có Wifi, thời gian là nghiên cứu bất kỳ tính thời điểm bất đầu và công nghệ như là vi điều khiển ESP32, cảm biến MPU6050, MAX30102, GPS Neo6M, OLED SSD1306 và nền tảng Blynk.

2.7 Phương pháp nghiên cứu

Phương pháp phân tích lý thuyết: Nghiên cứu tài liệu về IoT, cảm biến, thuật toán phát hiện té ngã, đo nhịp tim và SpO2.

Phương pháp thực nghiệm: Thiết kế, lập trình và thử nghiệm hệ thống trên ESP32.

Phương pháp tổng hợp: Đánh giá kết quả vừa mới thử nghiệm và tìm cách tối ưu hóa hệ thống.

2.8 Bố cục đề tài

Đề tài ngoài phần mở đầu, kết luận và tài liệu tham khảo thì nội dung chính của đề tài sẽ bao gồm 4 chương:

Chương 1: Giới thiệu đề tài

Chương 2: Phân tích yêu cầu

Chương 3: Những kiến thức liên quan

Chương 4: Ứng dụng

PHẦN 2: NỘI DUNG

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Tổng quan IoT trong y tế

IoT hay Internet Vạn Vật đề cập đến mạng lưới tập hợp các thiết bị thông minh và công nghệ tạo điều kiện thuận lợi cho hoạt động giao tiếp giữa thiết bị và đám mây cũng như giữa các thiết bị với nhau. Trong y tế, nền tảng IoT đã tạo ra một cuộc cách mạng mới, cung cấp các giải pháp hỗ trợ giám sát sức khỏe từ xa, phát hiện những tình trạng khẩn cấp và cải thiện chất lượng để có thể chăm sóc người lớn tuổi một cách tốt nhất.

Ứng dụng của IoT trong chăm sóc người cao tuổi mang lại giá trị thiết thực:

Giám sát sức khỏe liên tục: Theo dõi thông qua nhịp tim, nồng độ oxy trong máu theo thời gian thực tế, giúp để phát hiện sớm nhất dấu hiệu bất thường.

Phát hiện và cảnh báo ngã: Với những cảm biến, hệ thống có thể nhận diện trường hợp ngã đột ngột và gửi đến người thân hoặc nhân viên y tế, tránh tình trạng rủi ro xấu.

Định vị GPS hỗ trợ khẩn cấp: Tích hợp vị trí hiện tại, để có thể xác định vị trí trong thực tế và để có tìm kiếm người lớn tuổi nhanh chóng để ứng cứu.

1.2 Tổng quan về hệ thống phát hiện khẩn cấp

Phát hiện ngã (Lá chắn an toàn trước hiểm họa): Sử dụng cảm biến MPU6050 cùng đó là những yếu tố liên quan gia tốc và con quay hồi chuyển. Nhận dạng tình trạng ngã thông qua ngưỡng biên độ và thay đổi góc.

Giám sát sức khỏe (Người bác sĩ âm thầm): Sử dụng cảm biến MAX30102 được ứng dụng việc đo nhịp tim thông qua tín hiệu về hồng ngoại và SpO2.

Định vị GPS (Ánh sáng dẫn lối trong cơn khủng hoảng): Sử dụng module GPS Neo6M V2 khi xảy ra sự cố có thể cung cấp vị trí người dùng chỉ trong vài giây.

1.3 Giới thiệu về nền tảng Blynk hỗ trợ khía cạnh đề tài.

Blynk là một nền tảng IoT mạnh mẽ, được ứng dụng kết nối phần cứng với ứng dụng di động hoặc web, tạo thuận lợi người dùng có thể giám sát được. Nền tảng này kết nối dễ dàng, giao diện người dùng tùy chỉnh, hỗ trợ thời gian thực và tích hợp thông báo sự kiện.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU

2.1 Giới thiệu yêu cầu

Hệ thống phát hiện khẩn cấp dành cho người già được xây dựng và phát triển do nhu cầu thiết thực trong đời sống thực tế, được phân tích rõ ràng qua nhiều khía cạnh:

Phát hiện ngã: Phát hiện có thể đúng ít nhất 95% trường hợp ngã trong đời sống thực tế và chỉ hy hữu khoảng 5% cho trường hợp đặc biệt. Điều này cũng vô cùng đảm bảo rằng hạn chế nhiều nhất vấn đề nguy hiểm. Thời gian mà cảm biến phản hồi dưới 1 giây kích hoạt cảnh báo như buzzer và thông qua Blynk)

Giám sát sức khỏe: Chủ yếu đây nhóm thực hiện nghiên cứu hai khía cạnh là nhịp tim và SpO2 với mục tiêu là mạch lạc và liên tục. Hiển thị này lên màn hình OLED và hiển thị lên Web.

Định vị GPS: Lấy tọa độ bao gồm kinh độ và vĩ độ và sẽ có thể xảy một ít sai số phụ thuộc vào tín hiệu. Điều này xảy ra khi ngã hoặc nhấn nút khẩn cấp.

Cảnh báo khẩn cấp: Gửi những thông báo liên quan qua Blynk, email và xDroid và thời gian phản hồi cho vấn đề sự cố lên tới 5 giây sau khi vấn đề được phát hiện.

Tính ổn định và dự phòng: Sẽ kết nối Wifi và Blynk, nếu mất kết nối thì chỉ cần 5 giấy sẽ tự kết nối lại. Sẽ còn dự phòng là xDroid nếu xảy ra trường hợp xấu nhất.

2.2 Sơ đồ nguyên lý

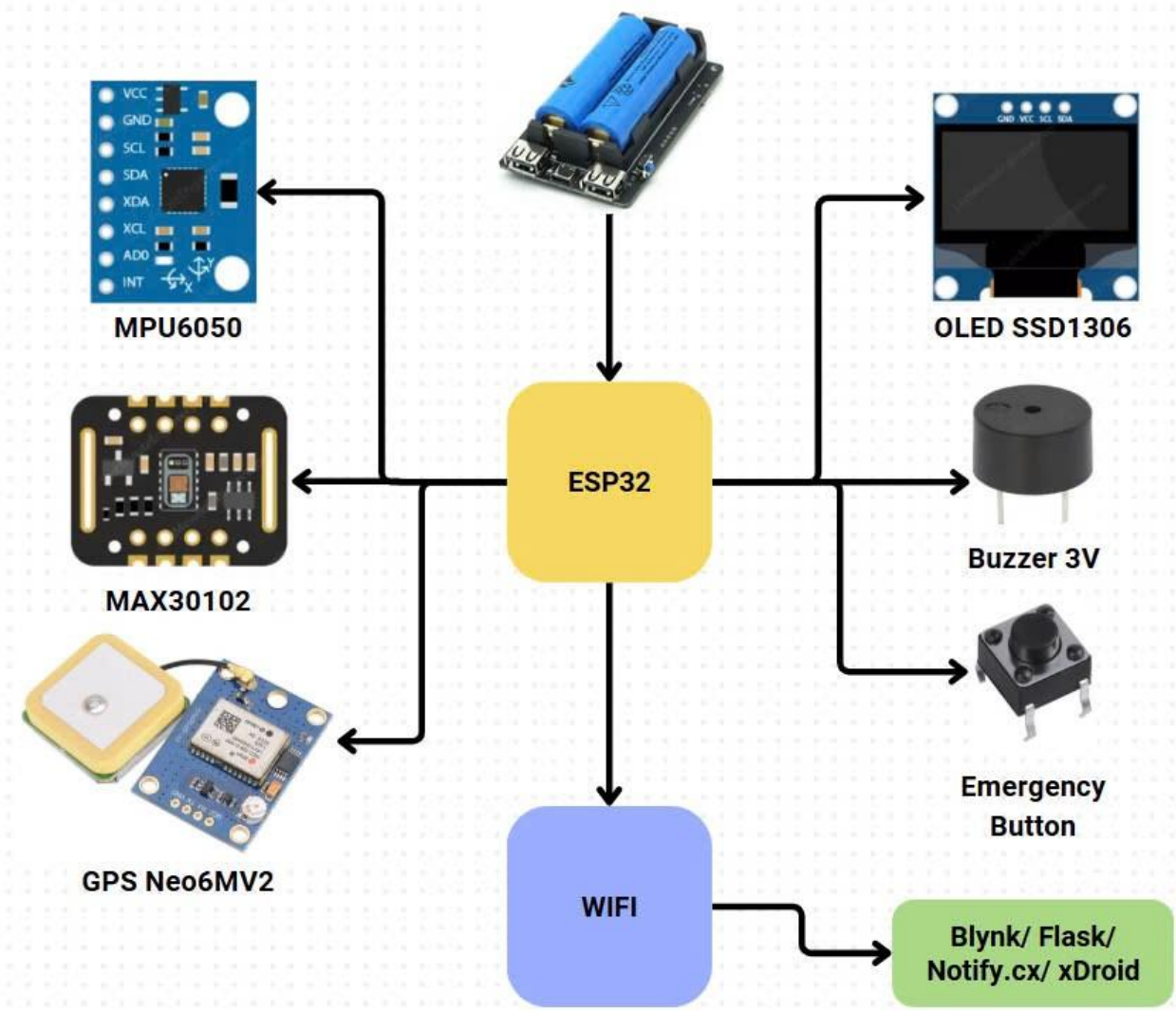
Trung tâm điều khiển: ESP32 quản lý toàn bộ hoạt động, xử lý dữ liệu từ cảm biến và gửi thông tin qua WiFi.

Cảm biến và thiết bị đầu vào: MPU6050 là cảm biến gia tốc và con quay hồi chuyển, kết nối qua I2C (SDA, SCL), phát hiện ngã; MAX30102 là cảm biến quang học, kết nối qua I2C, đo nhịp tim và SpO2; GPS Neo6M V2 là Module GPS, kết nối qua UART (RX=16, TX=17), cung cấp tọa độ.

Nút khẩn cấp: Kết nối qua GPIO 4 (INPUT_PULLUP), kích hoạt cảnh báo thủ công.

Thiết bị đầu ra: OLED SSD1306 là màn hình hiển thị (128x64 pixel), kết nối qua I2C, hiển thị nhịp tim, SpO2, thời gian, và trạng thái hệ thống; Buzzer là kết nối qua GPIO 15, phát âm thanh cảnh báo khi phát hiện ngã hoặc nút khẩn cấp được nhấn.

Giao tiếp phần mềm: Blynk là nền tảng gửi dữ liệu (nhịp tim, SpO2, GPS) và thông báo qua WiFi; Flask server là nhận dữ liệu qua endpoint /update_data (mỗi 2 giây), lưu trữ và hiển thị trên web; API notify.cx/xDroid gửi email và tin nhắn dự phòng khi Blynk không hoạt động.



Hình 2.1: Sơ đồ khối nguyên lý

2.3 Nguyên tắc làm việc

Hệ thống hoạt động dựa trên các nguyên tắc sau đảm bảo hiệu quả và độ tin cậy:

Phát hiện ngã:

+ Cảm biến MPU6050 để đo gia tốc (Acx, AcY, AcZ) và con quay hồi chuyển (GyX, GyY, GyZ).

```
int16_t AcX, AcY, AcZ, Tmp, GyX, GyY, GyZ;
```

+ Tính độ lớn của vector gia tốc tổng hợp từ ba trục ax, ay, az (amplitude): $\sqrt{(ax^2 + ay^2 + az^2)}$

```
float raw_amplitude = pow(pow(ax, 2) + pow(ay, 2) + pow(az, 2), 0.5);
```

+ Tính độ lớn của vector tốc độ góc, tức tổng tốc độ góc tính theo cả ba trục: sqrt $(gx^2 + gy^2 + gz^2)$ với sqrt là căn bậc hai

```
angleChange = pow(pow(gx, 2) + pow(gy, 2) + pow(gz, 2), 0.5);
```

+ Áp dụng ba ngưỡng để xác định tình trạng ngã:

Trigger 1: Khi amplitude ≤ 2. Đây được xem là một trạng thái tĩnh.

```
if (amplitude <= 2 && trigger2 == false)
```

Trigger 2: Khi amplitude ≥ 12. Đây là chuyển động mạnh.

```
if (trigger1) {
  trigger1count++;
  if (amplitude >= 12) {
```

Trigger 3: Độ lớn của vector tốc độ góc trong 30 đến 400 , sau đó trở về 0 đến 10 thì xác nhận đã ngã. Giải pháp này chỉ xảy ra khi mà Trigger 2 gán thành giá true.

```
  if (angleChange >= 30 && angleChange <= 400) { //if orientation cha
    trigger3 = true; trigger2 = false; trigger2count = 0;
    Serial.println(angleChange);
    Serial.println("TRIGGER 3 ACTIVATED");
  }
}
if (trigger3 == true) {
  trigger3count++;
  if (trigger3count >= 10) {
    angleChange = pow(pow(gx, 2) + pow(gy, 2) + pow(gz, 2), 0.5);
    Serial.println(angleChange);
    if ((angleChange >= 0) && (angleChange <= 10)) { //if orientatio
      fall = true; trigger3 = false; trigger3count = 0;
```

Hình 2.2: Mô phỏng trigger3 xảy ra

+ Phát hiện ngã dựa trên buzzer, hiển thị OLED và thông báo nền tảng. Bật buzzer phát ra âm thanh. Tiếp theo, in ra màn hình OLED dòng chữ *"FALL DETECTION"* và bắt đầu gửi cảnh báo, cuối cùng gửi qua nền tảng Blynk hoặc gửi trường hợp dự trù để có thể nhận được thông báo.

```
// BẬT BUZZER
digitalWrite(BUZZER_PIN, HIGH);
Serial.println("Buzzer ring!");

// Hien thi len oled
display.clearDisplay();
display.setCursor(0,0);
display.println("FALL DETECTION");
display.println("SENDING ALERT...");
display.display();

// Kiểm tra Blynk
if (Blynk.connected()) {
  Serial.println("Blynk is connected -> Using Blynk");

  Blynk.logEvent("fall_detected_notification", "User has fallen!");

  if ((lastEmailTime == 0) || (currentTime - lastEmailTime >= EMAIL_INTERVAL)) {
    Blynk.logEvent("fall_detected_email", "User has fallen!");
    lastEmailTime = currentTime;
  }
} else {
  Serial.println("Blynk not connected -> Using xDroid + notify.cx");
  serverName = "https://xdroid.net/api/message?k=k-e0e76f13322c&t=HELP!&c=FALL_DETECTION&u=http://google.com";
  sendCustom("true");
  serverName = "";

  if ((lastEmailTime == 0) || (currentTime - lastEmailTime >= EMAIL_INTERVAL)) {
    sendEmail();
    lastEmailTime = currentTime;
  }
}
```

Hình 2.3: Code mô phỏng thông báo

Giám sát sức khỏe:

+ Cảm biến MAX30102 đo tín hiệu hồng ngoại (IR) và ánh sáng đỏ để tính nhịp tim và SpO2. Những cấu hình và thông số liên quan:

```
// Khởi tạo MAX30102
if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
  Serial.println("Không tìm thấy MAX30102");
  while (1);
}
// Các tùy chọn sau có thể tự điều chỉnh
byte ledBrightness = 0x7F; // Độ sáng khuyến nghị = 127, Các tùy chọn: 0 = Tắt đến 255 = 50mA, để HEX
byte sampleAverage = 4;    // Số mẫu trung bình: 1, 2, 4, 8, 16, 32
byte ledMode = 2;          // Chế độ LED: 1 = Chỉ đỏ (nhịp tim), 2 = Đỏ + Hồng ngoại (SpO2)
int sampleRate = 800;      // Tốc độ mẫu: 50, 100, 200, 400, 800, 1000, 1600, 3200
int pulseWidth = 411;      // Độ rộng xung: 69, 118, 215, 411
int adcRange = 16384;      // Phạm vi ADC: 2048, 4096, 8192, 16384
```

Hình 2.4: Code khởi tạo MAX30102

+ Nhịp tim: Dựa trên thời gian giữa các nhịp đập (delta). Tính BPM:

$$beatsPerMinute = 60 / (delta / 1000.0)$$

Giá trị trung bình lấy từ 4 mẫu.

```
// Cập nhật hiển thị BPM & SpO2 khi không có ngã
if (!fall && !fallAlarmActive && !emergencyTriggered) {
  long irValue = particleSensor.getIR();    // Đọc giá trị hồng ngoại, dùng để kiểm tra có ngón tay đặt lên cảm biến hay không
  // Kiểm tra xem có đặt ngón tay lên không
  if (irValue > FINGER_ON ) {
    // Kiểm tra nhịp tim, đo nhịp tim
    if (checkForBeat(irValue) == true) {
      // Hiệu ứng hiển thị nhịp tim
      display.clearDisplay();                    // Xóa màn hình
      display.drawBitmap(0, 0, logo3_bmp, 32, 32, WHITE); // Hiển thị biểu tượng nhịp tim lớn
      display.setTextSize(2);                    // Thiết lập kích thước chữ
      display.setTextColor(WHITE);               // Màu chữ
      display.setCursor(42, 10);                 // Đặt vị trí con trỏ
      //display.print(beatAvg); display.println(" BPM"); // Hiển thị giá trị nhịp tim
      display.drawBitmap(0, 35, O2_bmp, 32, 32, WHITE);    // Hiển thị biểu tượng oxy
      display.setCursor(42, 40);                 // Đặt vị trí con trỏ
      // Hiển thị giá trị SpO2
      if (beatAvg > 30) display.print(String(ESpO2) + "%");
      else display.print("---- %" );
      display.display();                         // Cập nhật hiển thị
```

```
    long delta = millis() - lastBeat;          // Tính khoảng cách thời gian giữa các nhịp tim
    lastBeat = millis();
    beatsPerMinute = 60 / (delta / 1000.0);     // Tính nhịp tim theo phút
    if (beatsPerMinute < 255 && beatsPerMinute > 20) {
      // Nhịp tim phải trong khoảng 20 đến 255
      rates[rateSpot++] = (byte)beatsPerMinute; // Lưu giá trị nhịp tim vào mảng
      rateSpot %= RATE_SIZE;
      beatAvg = 0;                             // Tính giá trị trung bình
      for (byte x = 0 ; x < RATE_SIZE ; x++) beatAvg += rates[x];
      beatAvg /= RATE_SIZE;
    }
  }
```

Hình 2.5: Code tính toán nhịp tim

+ SpO2: Tính tỷ lệ R với

$$R = \frac{\sqrt{\text{Tổng bình phương sai số giá trị hồng ngoại IR}}}{\text{Trung bình giá trị hồng ngoại IR}} : \frac{\sqrt{\text{Tổng bình phương sai số giá trị ánh sáng đỏ}}}{\text{Trung bình giá trị ánh sáng đỏ}}$$

Sau đó áp dụng công thức tính SpO2:

$$SpO2 = -23.3 * (R - 0.4) + 120$$

```
// Đo SpO2
uint32_t ir, red;
double fred, fir;
particleSensor.check(); // Kiểm tra cảm biến, đọc tối đa 3 mẫu
if (particleSensor.available()) {
  i++;
  ir = particleSensor.getFIFOIR();    // Đọc giá trị hồng ngoại từ FIFO
  red = particleSensor.getFIFORed();    // Đọc giá trị ánh sáng đỏ từ FIFO
  //Serial.println("red=" + String(red) + ",IR=" + String(ir) + ",i=" + String(i));
  fir = (double)ir;                     // Chuyển đổi sang kiểu double
  fred = (double)red;                   // Chuyển đổi sang kiểu double
  aveir = aveir * frate + (double)ir * (1.0 - frate); // Tính trung bình giá trị IR qua bộ lọc thông thấp
  avered = avered * frate + (double)red * (1.0 - frate); // Tính trung bình giá trị đỏ qua bộ lọc thông thấp
  sumirrms += (fir - aveir) * (fir - aveir);  // Tính tổng bình phương sai số của giá trị IR
  sumredrms += (fred - avered) * (fred - avered); // Tính tổng bình phương sai số của giá trị đỏ

  if ((i % Num) == 0) {
    double R = (sqrt(sumirrms) / aveir) / (sqrt(sumredrms) / avered);
    SpO2 = -23.3 * (R - 0.4) + 120;
    ESpO2 = FSpO2 * ESpO2 + (1.0 - FSpO2) * SpO2; // Bộ lọc thông thấp cho SpO2 ước tính
    if (ESpO2 <= MINIMUM_SPO2) ESpO2 = MINIMUM_SPO2; // Chỉ báo khi ngón tay bị bỏ ra
    if (ESpO2 > 100) ESpO2 = 99.9;
    //Serial.print(",SPO2="); Serial.println(ESpO2);
    sumredrms = 0.0; sumirrms = 0.0; SpO2 = 0;
    i = 0;
  }
  particleSensor.nextSample();         // Xử lý xong mẫu này, chuyển sang mẫu tiếp theo
}
```

Hình 2.6: Code tính toán SpO2

+ Dữ liệu được hiển thị trên OLED và gửi đến Blynk (mỗi 1 giây) và Flask server (mỗi 2 giây).

Định vị GPS:

+ Module GPS Neo6M V2 đọc dữ liệu NMEA qua UART, sử dụng thư viện TinyGPSPlus.h để phân tích tọa độ (lat, lon).

+ Khi có sự cố (ngã hoặc nhấn nút khẩn cấp), hệ thống gửi tọa độ qua Blynk (widget V10) dưới dạng link Google Maps và bản đồ tĩnh từ Geoapify, đồng thời gửi đến Flask server.

```
void gpsTaskFunction(void * parameter) {
  Serial.println("GPS Task started");

  while(true) {
    // Đọc dữ liệu GPS trong 1 giây
    bool newData = false;
    unsigned long start = millis();

    while (millis() - start < 1000) {
      if (neogps.available()) {
        if (gps.encode(neogps.read())) {
          newData = true;
        }
      }
      // Cho phép các task khác thực thi
      vTaskDelay(1);
    }

    // Cập nhật dữ liệu GPS
    if (newData && gps.location.isValid()) {
      lat = gps.location.lat();
      lon = gps.location.lng();
      haveGPS = true;
    } else {
      haveGPS = false;
      Serial.println(F("No valid GPS data"));
      Serial.println();
    }

    // Đảm bảo có khoảng thời gian đủ trước khi bắt đầu chu kỳ đọc GPS mới
    vTaskDelay(10 / portTICK_PERIOD_MS);
  }
```

Hình 2.7: Code thực thi GPS

Cảnh báo khẩn cấp:

+ Kiểm tra trạng thái kết nối Blynk:

Nếu Blynk khả dụng: Gửi thông báo qua sự kiện (*fall_detected_notification, need_help*) và email (*fall_detected_email*) nếu thời gian giữa các email ≥ 20 giây.

Nếu Blynk không khả dụng: Chuyển sang Notify.cx (email) và xDroid (tin nhắn), kèm link Google Maps.

```
  // Kiểm tra Blynk
  if (Blynk.connected()) {
    Serial.println("Blynk is connected -> Using Blynk");

    Blynk.logEvent("fall_detected_notification", "User has fallen!");

    if ((lastEmailTime == 0) || (currentTime - lastEmailTime >= EMAIL_INTERVAL)) {
      Blynk.logEvent("fall_detected_email", "User has fallen!");
      lastEmailTime = currentTime;
    }
  } else {
    Serial.println("Blynk not connected -> Using xDroid + notify.cx");
    serverName = "https://xdroid.net/api/message?k=k-e0e76f13322c&t=HELP!&c=FALL_DETECTION&u=http://google.com";
    sendCustom("true");
    serverName = "";

    if ((lastEmailTime == 0) || (currentTime - lastEmailTime >= EMAIL_INTERVAL)) {
      sendEmail();
      lastEmailTime = currentTime;
    }
  }
}
```

Hình 2.8: Code kiểm tra Blynk

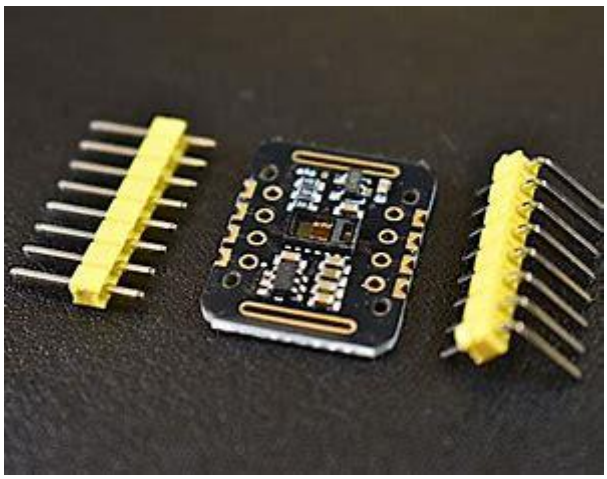
+ Buzzer được kích hoạt trong 10 giây khi phát hiện ngã hoặc 500ms khi nhấn nút khẩn cấp.

Quản lý kết nối: ESP32 tự động thử kết nối lại WiFi và Blynk mỗi 5 giây nếu mất tín hiệu, đảm bảo hệ thống hoạt động liên tục.

2.4 Lựa chọn giải pháp

Hình 2.9: ESP32

ESP32 là vi điều khiển 32-bit với lõi kép Ktensa LX6, hỗ trợ WiFi, mạnh mẽ, phù hợp cho IoT và có 34 chân GPIO hỗ trợ giao tiếp I2C, SPI, UART và ADC.



Hình 2.10: MAX30102

MAX30102 là cảm biến quang học tích hợp đèn LED đỏ và hồng ngoại, bộ quang thu và ADC 18-bit. Hỗ trợ cho việc giao tiếp qua I2C. Độ chính xác cao, hỗ trợ cả nhịp tim và SpO2.

Hình 2.11: MPU6050

MPU6050 là cảm biến MEMS 6 trục bao gồm 3 trục trong đo gia tốc và 3 trục trong con quay hồi chuyển. Giao tiếp thông qua I2C, độ phân giải 16-bit. Cung cấp dữ liệu gia tốc và con quay hồi chuyển, phù hợp phát hiện ngã.

Hình 2.12: Neo-6M V2

Neo-6M V2 là module GPS của u-blox, hỗ trợ định vị chính xác xấp xỉ gần 2.5m, giao tiếp UART và cung cấp dữ liệu NMEA(tọa độ, thời gian, tốc độ, số vệ tinh) Độ chính xác GPS tốt, dễ tích hợp.

Blynk là nền tảng IoT cung cấp giao diện người dùng trên ứng dụng di động, hỗ trợ gửi/nhận dữ liệu qua Virtual Pins, gửi thông báo sự kiện, và hiển thị bản đồ.

Flask server là framework Python nhẹ, dùng để xây dựng API RESTful nhận dữ liệu từ ESP32 qua HTTP POST. Server lưu trữ và phân tích dữ liệu sức khỏe, tọa độ, và trạng thái ngã.

Hình 2.13: OLED SSD1306

OLED SSD1306 là màn hình đơn sắc 128x64 pixel, giao tiếp I2C, tiêu thụ năng lượng thấp (~10-20mA) và hỗ trợ hiển thị văn bản, biểu tượng, và đồ họa đơn giản.

Hình 2.14: Battery 18650

Battery 18650 là nguồn 3.3V, dùng để cấp nguồn cho mô hình ESP32 vào chân 3.3V. Sử dụng chân cắm USB để cấp năng lượng sau khi hết pin.

CHƯƠNG 3: NHỮNG KIẾN THỨC LIÊN QUAN

3.1 Kiến thức liên quan phần cứng (chức năng cơ bản và thông số kĩ thuật)

ESP32 là vi điều khiển 32-bit mạnh mẽ, tích hợp Wifi và Bluetooth. Thứ nhất, điều khiển cảm biến như MAX30102, MPU6050 và GPS Neo-6M V2. Thứ hai, kết nối mạng gửi đến Blynk, Flask Server và các API khác. Cuối cùng là xử lý dữ liệu thô từ môi trường thực tế thông qua các dữ liệu và để tính toán những thông số như nhịp tim, SpO2 và tình trạng té ngã. Những thông số liên quan:

I2C (cho MAX30102, MPU6050, OLED SSD1306):

+ SDA: GPIO21 (mặc định cho I2C trên ESP32).

+ SCL: GPIO22 (mặc định cho I2C trên ESP32).

UART (cho GPS Neo-6M V2):

+ RX (GPS): GPIO16 (UART1 RX).

+ TX (GPS): GPIO17 (UART1 TX).

GPIO (cho nút khẩn cấp và buzzer):

+ Nút khẩn cấp: GPIO4 (INPUT_PULLUP).

+ Buzzer: GPIO15 (OUTPUT).

Nguồn:

+ 3.3V và GND: Cung cấp nguồn cho các cảm biến và module.

+ 5V (VIN): Có thể sử dụng để cấp nguồn cho ESP32 qua USB hoặc nguồn ngoài.

MAX30102 là cảm biến quang học sử dụng ánh sáng đỏ và hồng ngoại để đo nhịp tim và nồng độ oxy trong máu (SpO2), được tích hợp giám sát sức khỏe. Đo nhịp tim bằng cách phát hiện sự thay đổi trong lưu lượng máu. Đo SpO2 dựa trên sự hấp thụ ánh sáng khác nhau giữa hemoglobin có oxy và không có oxy. Thông số kĩ thuật liên quan:

I2C:

+ SDA: Kết nối với GPIO21 của ESP32.

+ SCL: Kết nối với GPIO22 của ESP32.

Nguồn:

+ VIN: 3.3V từ ESP32.

+ GND: Kết nối với GND của ESP32.

MPU6050 là cảm biến 6 trục, đo gia tốc (3 trục) và con quay hồi chuyển (3 trục), được sử dụng để phát hiện chuyển động và thay đổi tư thế. Phát hiện té ngã dựa trên sự thay đổi đột ngột của gia tốc và góc nghiêng. Cung cấp dữ liệu để tính toán góc xoay và biên độ chuyển động. Thông số kĩ thuật liên quan đến:

I2C:

+ SDA: Kết nối với GPIO21 của ESP32.

+ SCL: Kết nối với GPIO22 của ESP32.

Nguồn:

+ VCC: 3.3V từ ESP32.

+ GND: Kết nối với GND của ESP32.

Module GPS Neo-6M V2 cung cấp dữ liệu vị trí (kinh độ, vĩ độ), tốc độ và thời gian thông qua giao thức NMEA, hỗ trợ định vị trong các tình huống khẩn cấp. Xác định vị trí chính xác của người dùng để gửi cảnh báo kèm tọa độ. Cung cấp thời gian và ngày giờ, được điều chỉnh theo múi giờ Việt Nam (UTC+7). Thông số kĩ thuật liên quan:

UART:

+ TX (GPS): Kết nối với GPIO16 (UART1 RX) của ESP32.

+ RX (GPS): Kết nối với GPIO17 (UART1 TX) của ESP32.

Nguồn:

+ VCC: 3.3V hoặc 5V từ ESP32.

+ GND: Kết nối với GND của ESP32.

Màn hình OLED SSD1306 là màn hình đơn sắc 128x64 pixel, sử dụng giao thức I2C để hiển thị thông tin trực quan. Hiển thị nhịp tim, SpO2, ngày giờ và trạng thái hệ

thống (WiFi, Blynk, cảnh báo). Cung cấp giao diện trực quan với biểu tượng nhịp tim và oxy.

I2C:

+ SDA: kết nối với GPIO21 của ESP32.

+ SCL: kết nối với GPIO22 của ESP32.

Nguồn:

+ VCC: 3.3V từ ESP32.

+ GND: kết nối với GND của ESP32.

Blynk là nền tảng IoT cho phép điều khiển và giám sát thiết bị từ xa thông qua ứng dụng di động hoặc web. Gửi thông báo khi phát hiện té ngã hoặc nhấn nút khẩn cấp. Hiển thị dữ liệu nhịp tim, SpO2 và bản đồ GPS trên giao diện Blynk. Cho phép bật/tắt gửi dữ liệu nhịp tim và SpO2 từ xa. Hoạt động qua kết nối WiFi của ESP32

Flask là framework Python nhẹ để xây dựng API nhận dữ liệu từ ESP32, hỗ trợ lưu trữ và phân tích dữ liệu sức khỏe. Nhận dữ liệu nhịp tim, SpO2, trạng thái ngã và vị trí GPS từ ESP32. Hiển thị dữ liệu trên giao diện web (như bạn yêu cầu ngày 18/04/2025). Hỗ trợ mở rộng như lưu trữ dữ liệu vào cơ sở dữ liệu hoặc tích hợp với các dịch vụ khác. H oạt động qua kết nối HTTP với ESP32.

3.2 Kiến thức liên quan phần mềm (kiến thức áp dụng và tư duy lập trình)

Nguyên lý phát hiện té ngã: Cảm biến MPU6050 đây là cảm biến 6 trục, đo gia tốc (AcX, AcY, AcZ) và con quay hồi chuyển (GyX, GyY, GyZ). *Té ngã được phát hiện qua các dấu hiệu:*

+ Gia tốc bất thường: Một cú ngã thường bắt đầu bằng trạng thái tĩnh (gia tốc thấp), sau đó là gia tốc lớn (do va chạm) và kết thúc ở trạng thái tĩnh mới (nằm trên mặt đất).

+ Thay đổi góc nghiêng: Tư thế cơ thể thay đổi đột ngột (góc nghiêng lớn) và duy trì ở góc thấp khi người nằm xuống.

Thuật toán 3 giai đoạn (trigger1, trigger2, trigger3) với giá trị g=9.8m/s2 được tính bằng công thức:

$$\sqrt{ax^2 + ay^2 + az^2}.$$

+ *Trigger1:* Độ lớn biên độ gia tốc thấp (< 0.4g) cho thấy trạng thái tĩnh.

+ *Trigger2:* Độ lớn biên độ gia tốc cao (> 3g) cho thấy va chạm.

+ *Trigger3:* Độ lớn của vector tốc độ góc thay đổi lớn từ (30-400) sau đó ổn định ở góc thấp (0-10) xác nhận té ngã.

Quản lý thời gian thực: Sử dụng hàm millis() để quản lý cập nhật thời gian ổn định và hoạt động thực. Có nghĩa việc đọc dữ liệu MPU6050 mỗi 100ms đảm bảo việc đọc và cập nhật liên tục. Gửi đến Flask server mỗi 2 giây được minh họa:

```
// Gửi dữ liệu đến Flask server
if (currentTime - lastServerUpdate >= SERVER_INTERVAL) {
  sendDataToServer();
  lastServerUpdate = currentTime;
}
```

Hình 3.1: Code quản lý thời gian thực

Kiểm tra GPS mỗi 1 giây. Từ yếu tố quản lý thời gian ta có thể nhận được 2 lợi ích lớn là tránh tình trạng chặn code và tối ưu hóa thời gian, tăng hiệu suất.

Xử lý dữ liệu GPS và điều chỉnh múi giờ:

```
void gpsTaskFunction(void * parameter) {
  Serial.println("GPS Task started");

  while(true) {
    // Đọc dữ liệu GPS trong 1 giây
    bool newData = false;
    unsigned long start = millis();

    while (millis() - start < 1000) {
      if (neogps.available()) {
        if (gps.encode(neogps.read())) {
          newData = true;
        }
      }
      // Cho phép các task khác thực thi
      vTaskDelay(1);
    }

    // Cập nhật dữ liệu GPS
    if (newData && gps.location.isValid()) {
      lat = gps.location.lat();
      lon = gps.location.lng();
      haveGPS = true;
    } else {
      haveGPS = false;
      Serial.println(F("No valid GPS data"));
      Serial.println();
    }

    // Đảm bảo có khoảng thời gian đủ trước khi bắt đầu chu kỳ đọc GPS mới
    vTaskDelay(10 / portTICK_PERIOD_MS);
  }
```

Hình 3.2: Code xử lý dữ liệu GPS

```
void updateDateTime() {
  if (haveGPS) {
    int hour = gps.time.hour();
    int minute = gps.time.minute();
    int day = gps.date.day();
    int month = gps.date.month();
    int year = gps.date.year();

    hour = (hour + 7) % 24;

    if ((gps.time.hour() + 7) >= 24) {
      day++;

      // Handle month rollover
      int daysInMonth;
      if (month == 2) {
        // Handle leap year
        daysInMonth = ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) ? 29 : 28;
      } else if (month == 4 || month == 6 || month == 9 || month == 11) {
        daysInMonth = 30;
      } else {
        daysInMonth = 31;
      }

      if (day > daysInMonth) {
        day = 1;
        month++;

        // Handle year rollover
        if (month > 12) {
          month = 1;
          year++;
```

Hình 3.3: Code cập nhật thời gian

Hiển thị đồ họa trên OLED SSD1306: Sử dụng thư viện Adafruit_GFX.h và Adafruit_SSD1306.h để vẽ văn bản, biểu tượng bitmap, và cập nhật màn hình OLED. Quản lý giao diện người dùng với các biểu tượng tùy chỉnh (nhịp tim, oxy) để tăng tính trực quan.

```
// Hiển thị dữ liệu lên màn hình OLED
display.clearDisplay();                  // Xóa màn hình
display.drawBitmap(5, 1, logo2_bmp, 24, 21, WHITE); // Hiển thị biểu tượng nhịp tim nhỏ
display.setTextSize(2);                  // Thiết lập kích thước chữ
display.setTextColor(WHITE);             // Màu chữ
display.setCursor(42, 5);               // Đặt vị trí con trỏ
display.print(beatAvg); display.println(" BPM"); // Hiển thị giá trị nhịp tim
display.drawBitmap(0, 30, O2_bmp, 32, 32, WHITE); // Hiển thị biểu tượng oxy
display.setCursor(42, 32);               // Đặt vị trí con trỏ
// Hiển thị giá trị SpO2, chỉ khi nhịp tim trên 30 để tránh đo sai
if (beatAvg > 30) display.print(String(ESpO2) + "%");
else display.print("---- %" );

display.setTextSize(1);
display.setCursor(26, 56); // Position at the bottom of the screen
display.print(clockDate + " " + clockTime);
display.display();
} else {
// Xóa dữ liệu nhịp tim
for (byte rx = 0 ; rx < RATE_SIZE ; rx++) rates[rx] = 0;
beatAvg = 0; rateSpot = 0; lastBeat = 0;
// Xóa dữ liệu SpO2
avered = 0; aveir = 0; sumirrms = 0; sumredrms = 0;
SpO2 = 0; ESpO2 = 90.0;
```

Hình 3.4: Cập nhật màn hình OLED

Xử lý nút khẩn cấp và buzzer: Sử dụng GPIO để đọc trạng thái nút nhấn (INPUT_PULLUP) và điều khiển buzzer (OUTPUT). Xử lý sự kiện nhấn nút với thời gian giữ tối thiểu (debouncing) để tránh báo động giả. Kích hoạt buzzer như một tín hiệu cảnh báo âm thanh.

```
  // Kiểm tra nút nhấn
  if (digitalRead(EMERGENCY_BUTTON_PIN) == LOW && !buttonPressed) {
    buttonPressed = true;
    buttonPressStartTime = millis();
  }
  else if (digitalRead(EMERGENCY_BUTTON_PIN) == HIGH) {
    buttonPressed = false;
  }
  // Nếu nút đang được nhấn và đạt thời gian yêu cầu
  if (buttonPressed && !emergencyTriggered && (millis() - buttonPressStartTime >= 2000)) {
    emergencyTriggered = true;
    emergencyStartTime = millis();
    sendLocation();

    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(WHITE);
    display.setCursor(30, 5);
    display.println("Sending");
    display.setCursor(30, 35);
    display.println("Help");
    display.display();

    Serial.println("Emergency Button pressed for 2s, sending emergency alert.");
    Blynk.logEvent("need_help", "I need some help!");
    digitalWrite(BUZZER_PIN, HIGH);
  }

// Xử lý sau khi đã kích hoạt khẩn cấp
if (emergencyTriggered) {
  unsigned long currentTime = millis();
  // Tắt buzzer sau 500ms
  if (currentTime - emergencyStartTime >= 500 && currentTime - emergencyStartTime < 4500) {
    digitalWrite(BUZZER_PIN, LOW);
  }
  // Đặt lại trạng thái sau 3.5 giây
  if (currentTime - emergencyStartTime >= 3500) {
    emergencyTriggered = false;
  }
}
```

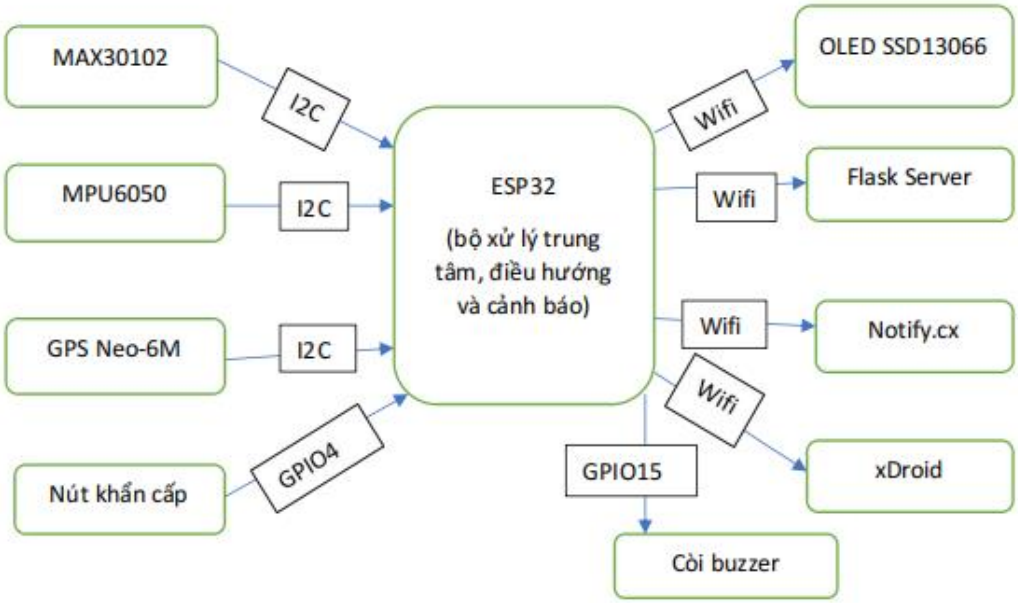
Hình 3.5: Code xử lý nút bấm

Giao tiếp HTTP và dự phòng thông báo: Sử dụng thành thạo thư viện HTTPClient.h để gửi các yêu cầu đến các API bên ngoài, đồng thời xây dựng lập trình dự phòng mất Wifi và cuối cùng thêm xử lý file JSON trong việc xử lý web.

Tối ưu hóa module, xử lý lỗi và dự phòng: Tái sử dụng các hàm độc lập trong source code như mpu_read(), sendLocation (), sendToFalskServer(), sendEmail() và sendCusstom() được sử dụng lại nhiều lần. Tách các hàm này ra riêng lẻ dễ dàng phát hiện trong việc xử lý lỗi, dễ bảo trì và phát triển. Dự đoán được những lỗi xảy ra và cung cấp giải pháp tối ưu dự phòng như mất Wifi, mất GPS hoặc cảm biến lỗi.

CHƯƠNG 4: ỨNG DỤNG

4.1 Sơ đồ khối ứng dụng



Hình 4.1: Sơ đồ khối ứng dụng

Từ sơ đồ khối ứng dụng chúng ta sẽ tiến hành phân tích chi tiết sau:

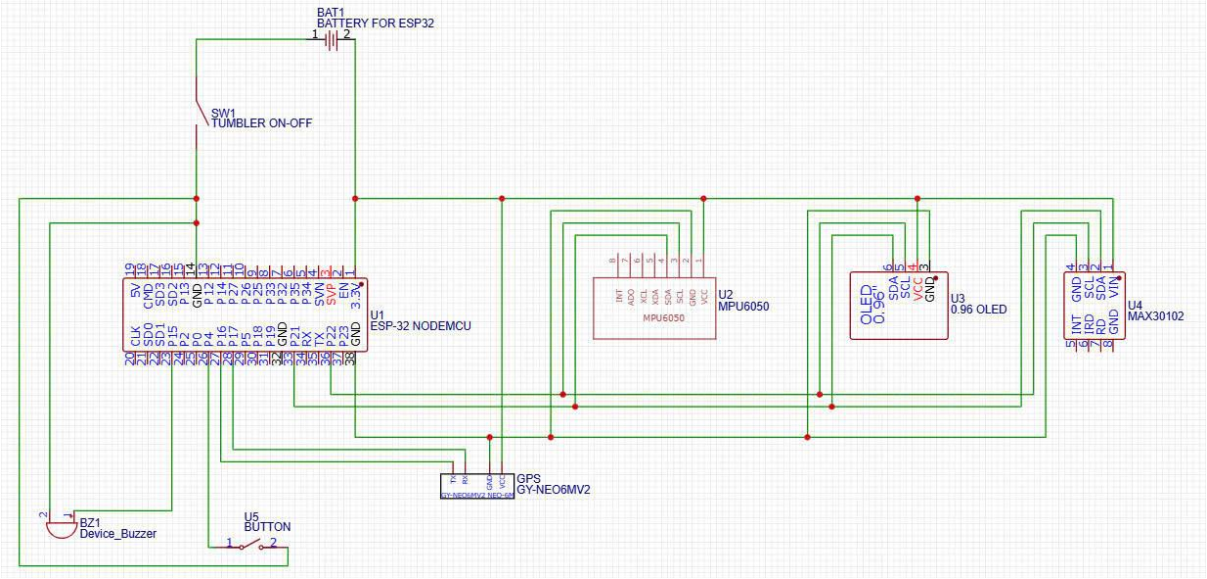
Dữ liệu đầu vào: Lấy những thông tin về những thông số của MAX30102, MPU6050, GPS Neo-6M, Buzzer để từ này xem phần thông tin thô xử lý.

Xử lý dữ liệu: Bộ xử lý trung tâm ESP32 đóng vai trò trái tim, điều khiển và xử lý dữ liệu thô, thành dữ liệu đạt yêu cầu người dùng từ đây phân luồng dữ liệu đến nền tảng khác nhau. Dữ liệu xử lý sẽ bao gômg tính toán nhịp tim, SpO2 từ ánh sáng đỏ và hông ngoại, dữ liệu trigger đánh giá tình trạng từ gia tốc và góc nghiêng. Cuối dùng dữ liệu GPS để lấy tọa độ và múi giờ hợp lý nhất.

Dữ liệu đầu ra: Lấy luồng dữ liệu đã được chia sẽ từ ESP32 sau đó thông báo các nền tảng Blynk, Flask Server, Notify và xDroid, ngoài ra thông báo tới màn hình OLED.

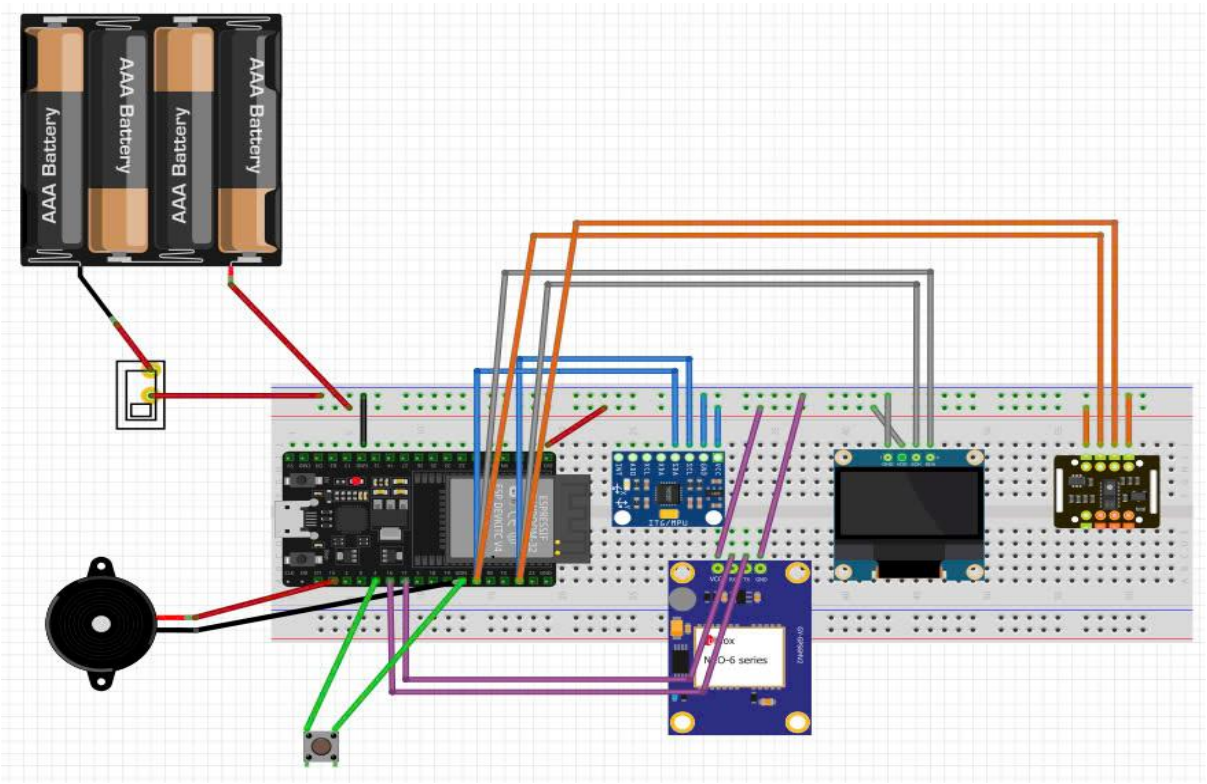
Phương thức giao tiếp chính: I2C, GPO15, GPO4 và Wifi.

4.2 Quy trình xây dựng



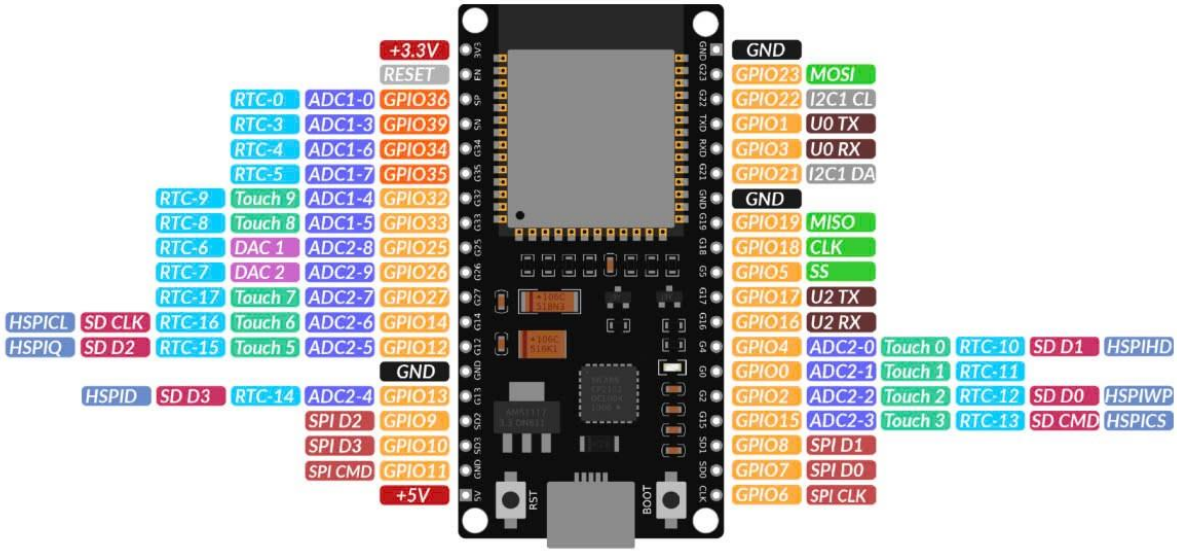
Hình 4.2: Mô hình ứng dụng thô cho bước 1

Bước 1: Xây dựng mô hình thô chi tiết, lên về phương pháp lắp mạch và định nghĩa về dữ liệu và thông số ban đầu. Chọn lựa những thiết bị phù hợp và giá cả hợp lý.



Hình 4.3: Mô hình mô phỏng cho bước 2

Bước 2: Xây dựng mô hình mô phỏng qua ứng dụng và định hình về yêu cầu cần làm, tìm hiểu chi tiết trên các thiết bị được sử dụng. Phân công thành viên tìm hiểu chi tiết các tiểu tiết của các thiết bị.



Hình 4.4: Module ESP32 cho bước 3

Bước 3: Phân tích định hình bộ xử lý trung tâm trước. Tìm hiểu về nhu cầu cần thiết liên quan lắp mạch và xây dựng. Phân tích chi tiết thành phần được sử dụng:

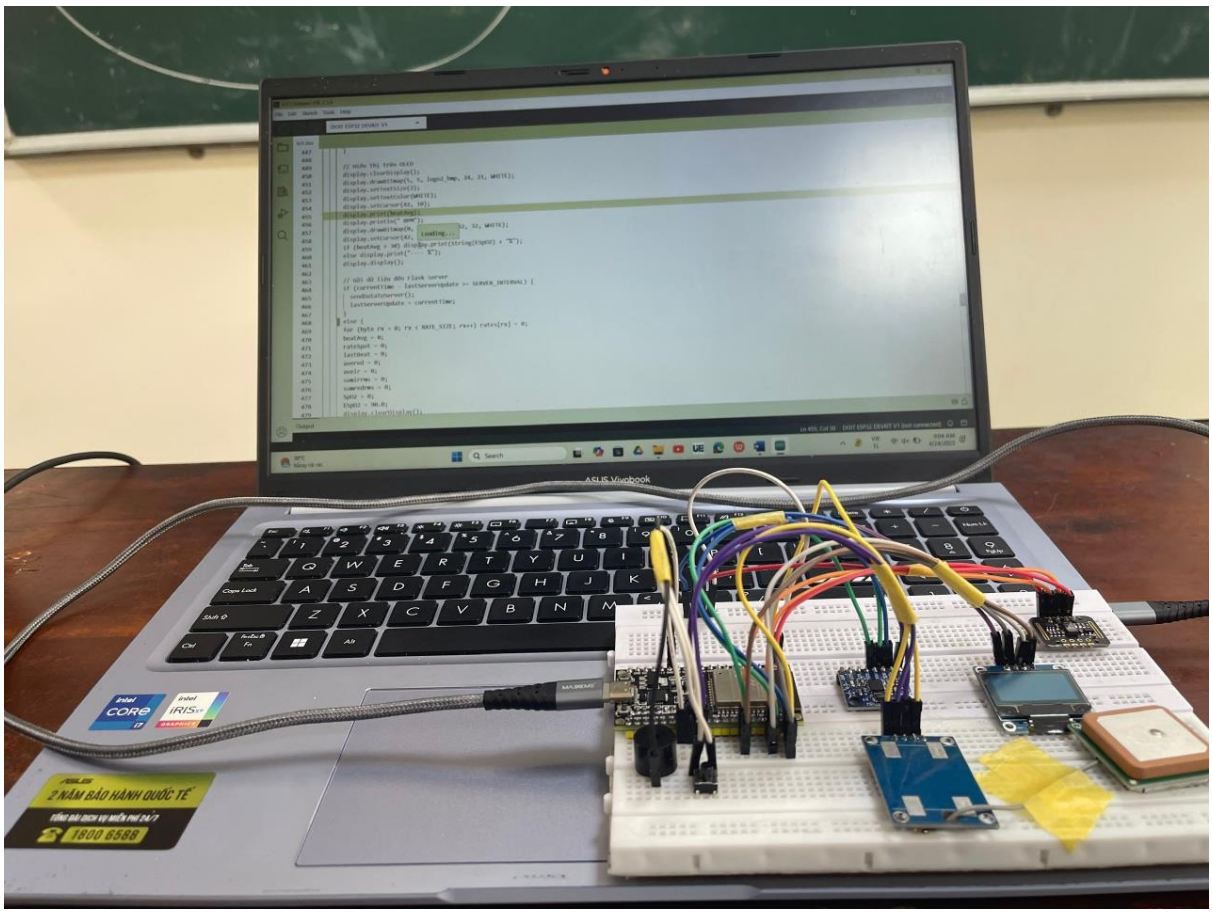
+ Module nào đó hoạt động ở điện áp 3.3V. Cắm chân VCC vào chân 3.3V

+ Chân GND của module thì cắm vào chân GND của esp32

+ Đối với module nào hoạt động dựa trên giao tiếp I2C, có nghĩa là có SCL với SCA, thì sử dụng 2 chân 21 và 22 của esp32 (SCL ở chân 22, SCA ở chân 21)

+ Đối với module có RX TX - nghĩa là GPS thì chân TX của module nối với chân RX của esp32 - hay là chân 16; RX của module nối với chân TX của esp32 - hay là chân 17

+ Còn đối với vấn đề khác như buzzer hay nút bấm thì chọn chân GPO tùy ý. Trong đây nhóm chọn chân GPO15 là chân buzzer, còn nút bấm là chân GPO4. Phù hợp để mạch đẹp và dễ nhìn hơn.



Hình 4.5: Thiết kế hoàn thiện cho bước 4

Bước 4: Xây dựng các linh kiện phụ phục vụ các chức năng như cảm biến, OLED, GPS. Trong đó dữ liệu được thiết kế như sau:

+ OLED sử dụng giao tiếp I2C, gồm hai chân chính là SDA và SCL lần lượt được nối với GPIO21 và GPIO22.

+ GPS sử dụng giao tiếp UART với TX và RX. Cụ thể chân 3 và chân 4 điều khiển bởi thư viện TinyGPS++.

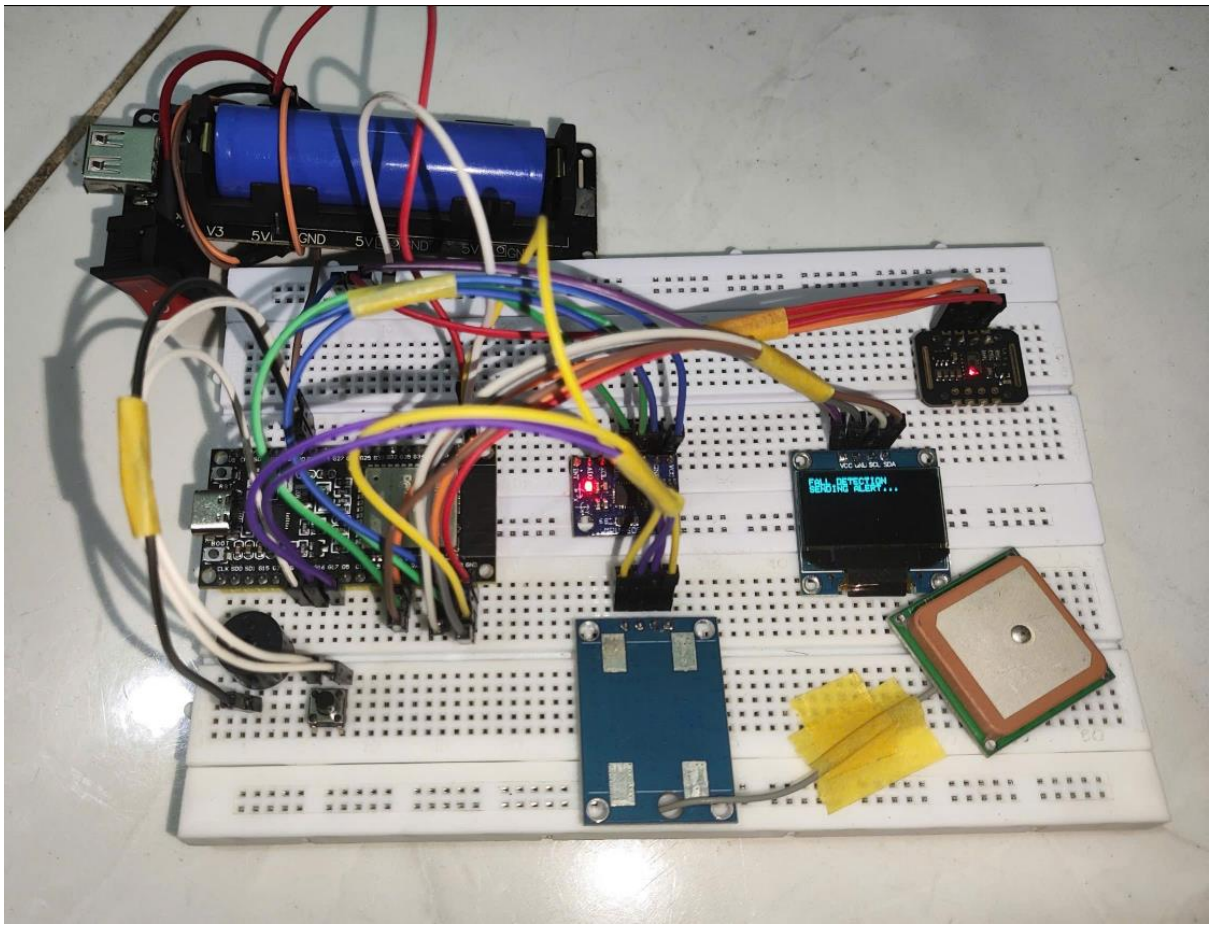
+ Nút nhấn cảnh báo (button) thường được cắm chân 6 và GND.

+ Còi (buzzer) thường được cắm chân 7 và GND.

+ MPU6050 chân GPIO21 với chân cảm biến SDA và chân GPIO22 với chân cảm biến SCL. Chân VIN là nguồn và chân GND. Với địa chỉ mặc định là 0x68.

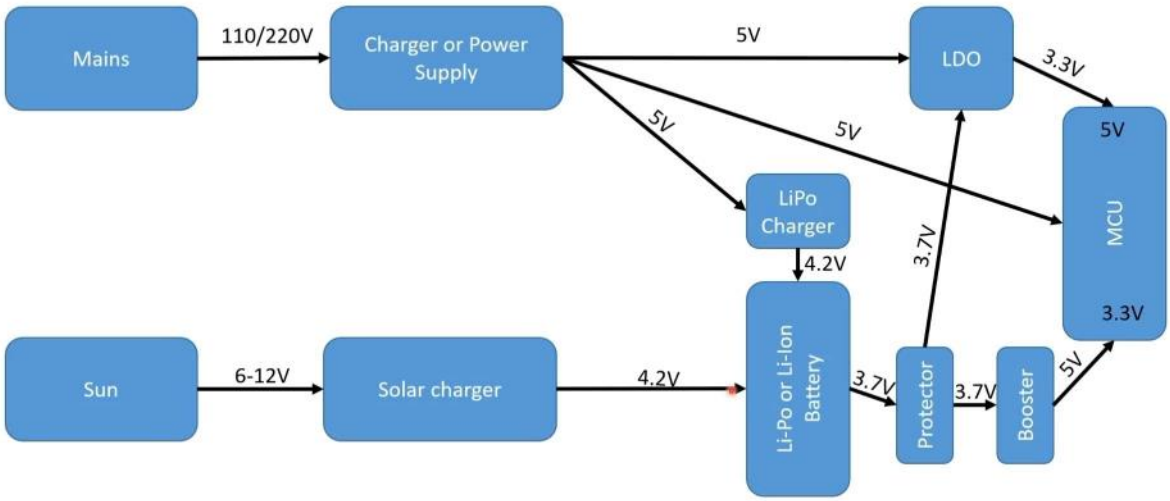
+ MAX30102 chân GPIO21 với chân cảm biến SDA và chân GPIO22 với chân cảm biến SCL. Chân VIN là nguồn và chân GND. Dùng chung bus I2C.

+ GPS Neo-6M chân D4 với chân cảm biến TX, chân D3 với chân cảm biến RX. Và chân Vcc là nguồn và chân GND. Dùng SoftwareSerial.



Hình 4.6: Sản phẩm mô phỏng bước 5

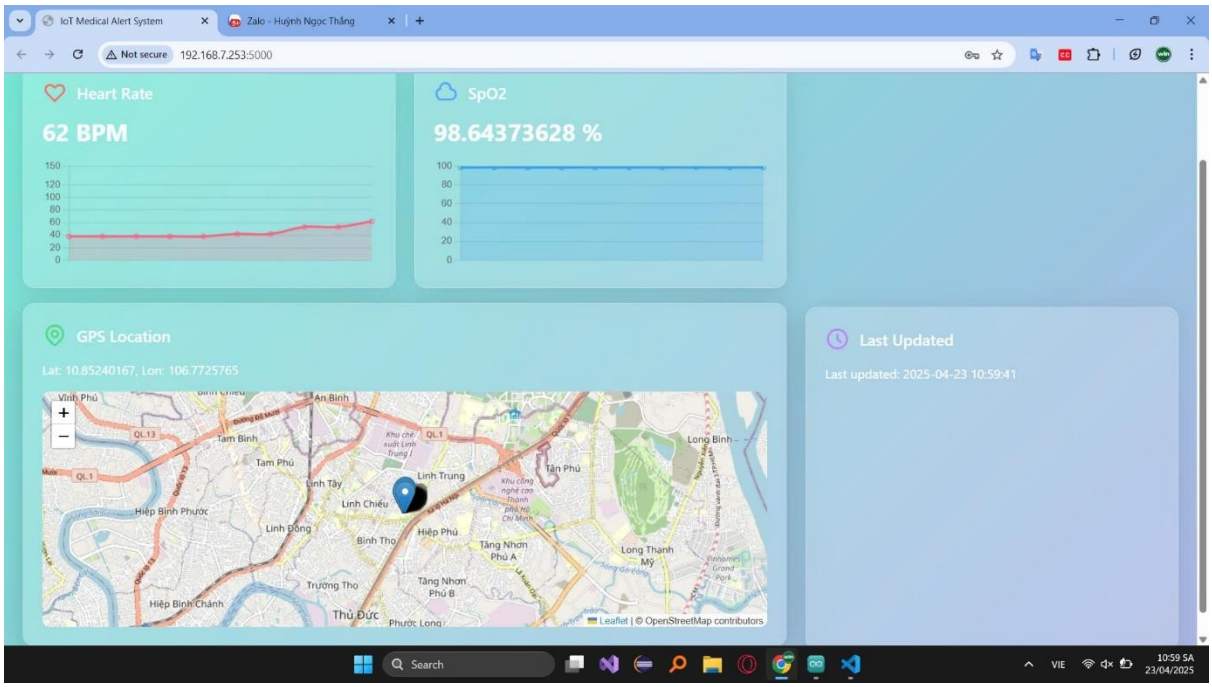
Bước 5: Cấp nguồn cho mạch và ESP32 bằng Battery 3.3V, hoàn thành sản phẩm tiện lợi và dễ sử dụng.



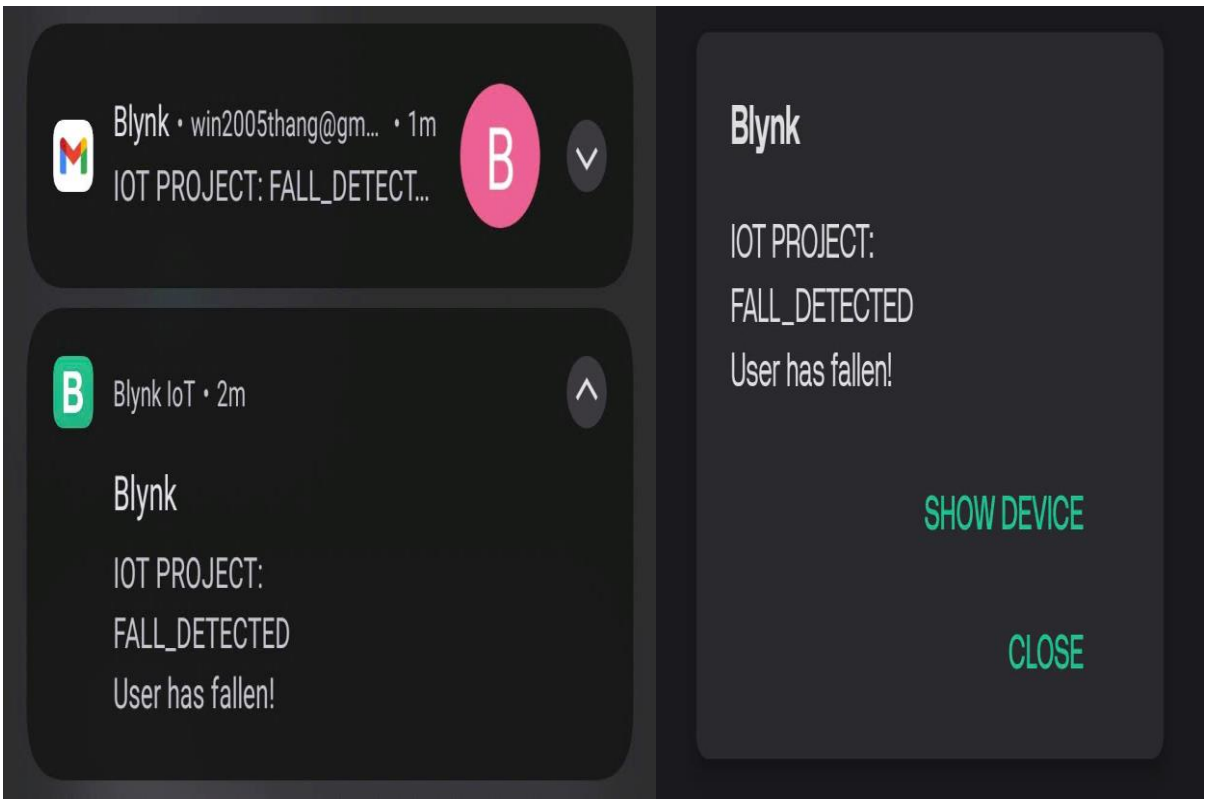
Hình 4.7: Phân tích cấu hình Battery

4.3 Những thành tựu đạt được, khó khăn và giải pháp

Giám sát sức khỏe thời gian thực được minh họa:



Hình 4.8: Hệ thống giám sát bằng Web



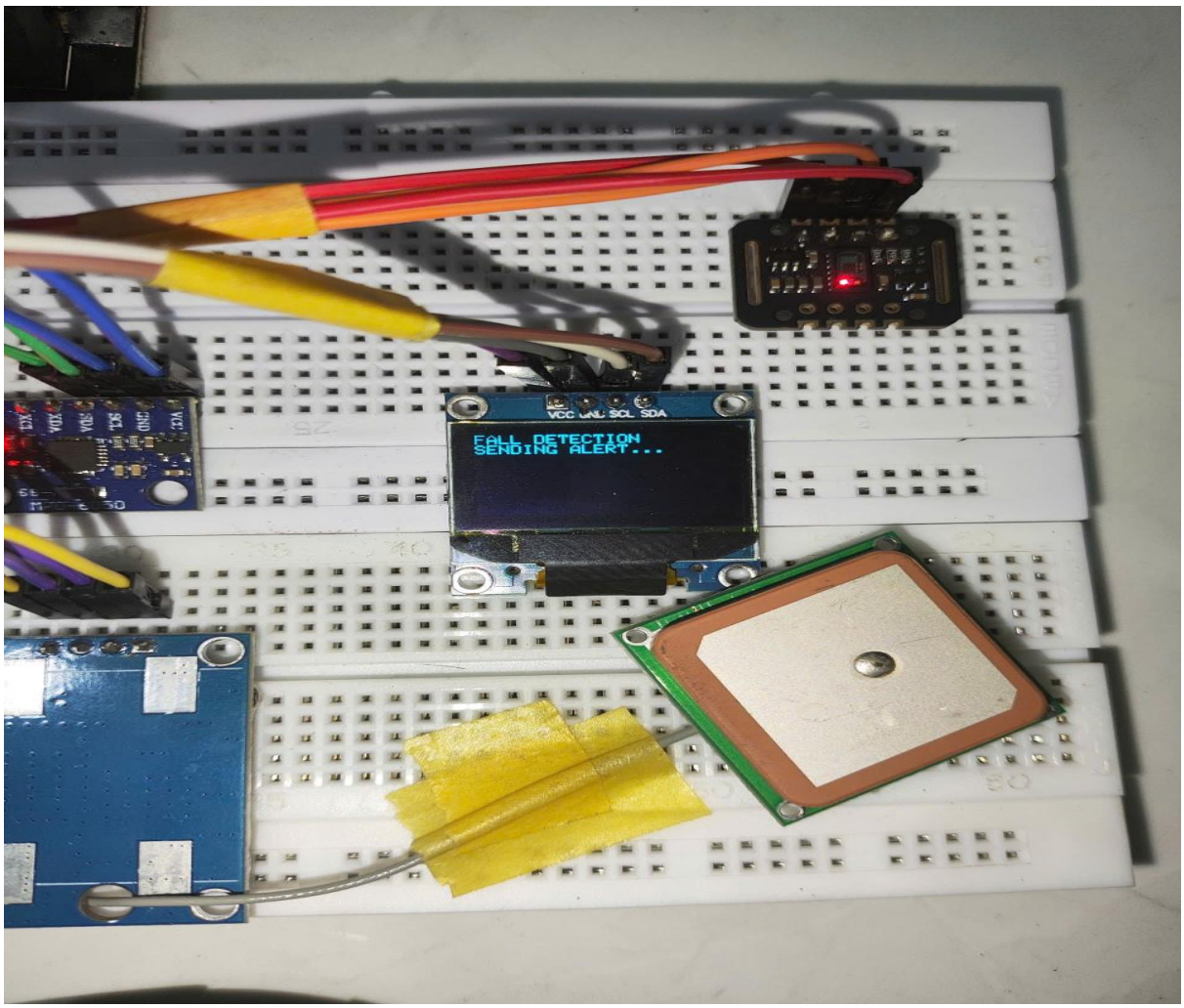
Hình 4.9: Hệ thống giám sát bằng Blynk

+ *Thành tựu:* Hệ thống sử dụng cảm biến MAX30102 để đo nhịp tim (BPM) và nồng độ oxy trong máu (SpO2) liên tục, cung cấp dữ liệu sức khỏe quan trọng. Dữ liệu được hiển thị trực quan trên màn hình OLED SSD1306 và gửi đến Blynk/Flask để người thân hoặc nhân viên y tế theo dõi từ xa. Thông qua cả hai nền tảng người thân hoặc các nhân viên y tế có thể đánh giá và xem xét tình hình có điều chỉnh phù hợp.

+ *Khó khăn:* ESP32 phải xử lý nhiều tác vụ đồng thời (đọc cảm biến, hiển thị OLED, gửi dữ liệu qua WiFi), dễ gây chậm trễ hoặc tắc nghẽn. Bộ nhớ hạn chế có thể dẫn đến lỗi nếu sử dụng quá nhiều thư viện hoặc dữ liệu JSON lớn. Khi Flask không khởi động có thể gây lãng phí thời gian khi gửi dữ liệu và yêu cầu truy cập.

+ *Giải pháp:* Giảm tần suất gửi dữ liệu đến Flask/Blynk (ví dụ: từ 1 giây xuống 5 giây) để giảm tải. Sử dụng ESP32-S3 hoặc các vi điều khiển mạnh hơn nếu cần xử lý nhiều tác vụ phức tạp. Có thể giải quyết Flask bằng cách phân luồng dữ liệu.

Phát hiện té ngã tự động được minh họa:



Hình 4.10: Cảnh báo trên OLED

+ *Thành tựu:* Sử dụng cảm biến MPU6050 để phát hiện té ngã thông qua thuật toán 3 giai đoạn (trigger1, trigger2, trigger3) dựa trên gia tốc và góc nghiêng. Khi phát hiện té ngã, hệ thống kích hoạt buzzer, hiển thị "FALL DETECTION" trên OLED, và gửi cảnh báo qua Blynk/Notify.cx/xDroid. Cảnh báo này được mô phỏng bằng âm thanh, nhạc chuông phù hợp.

+ *Khó khăn:* Thuật toán phát hiện té ngã dựa trên các ngưỡng cố định (0.4g, 3g, 30-400 độ) có thể không chính xác trong một số trường hợp, như ngã chậm, ngã trên bề mặt mềm, hoặc chuyển động bất thường (ví dụ: ngồi xuống nhanh). Cảm biến MPU6050 dễ bị nhiễu từ rung động hoặc sai lệch nếu không được hiệu chỉnh đúng.

+ *Giải pháp:* Sử dụng thuật toán 3 giai đoạn với biến đếm (trigger1count, trigger2count, trigger3count) để tránh báo động giả. Kết hợp dữ liệu từ nhiều cảm biến (ví dụ: kiểm tra nhịp tim từ MAX30102 sau khi phát hiện té ngã để xác nhận tình trạng).

Gửi cảnh báo kèm vị trí GPS được minh họa:

```
21:31:42.185 -> ▶ Location sent:
21:31:42.223 ->    Latest Location: 21:31 23/04/2025
21:31:42.223 ->    Link: https://www.google.com/maps/search/?api=1&query=10.856359,106.762047
21:31:42.223 ->    Img : https://maps.geoapify.com/v1/staticmap?style=osm-carto&width=600&height=400&center=lonlat:106.762047,10.856359&zoom=14&apiKey=dea92aad23494dc89d60e0d3b095ca76
21:31:42.261 -> FALL DETECTED
21:31:42.261 -> Buzzer ring!
21:31:42.261 -> Blynk is connected -> Using Blynk
21:31:42.467 -> === GPS Signal ===
21:31:42.467 -> Latitude:  10.856359
21:31:42.467 -> Longitude: 106.762047
21:31:42.467 -> Speed:     2.96 km/h
21:31:42.467 -> Altitude:  39 m
21:31:42.508 -> Satellites:4
```

Hình 4.11: Định vị GPS

+ *Thành tựu:* Module GPS Neo-6M cung cấp tọa độ (kinh độ, vĩ độ) và thời gian, được gửi kèm cảnh báo khi phát hiện té ngã hoặc nhấn nút khẩn cấp. Hệ thống gửi liên kết Google Maps và bản đồ tĩnh (Geoapify) qua Blynk, giúp người nhận xác định vị trí chính xác

+ *Khó khăn*: Do sử dụng nền tảng Wifi, tình trạng kém kết nối hoặc mất kết nối

+ *Giải pháp:* Cố gắng duy trì Wifi hiện trạng tốt nhất hoặc sử dụng thiết bị khác hỗ trợ thông Blutooth hoặc nền tảng kết nối khác dễ sử dụng.

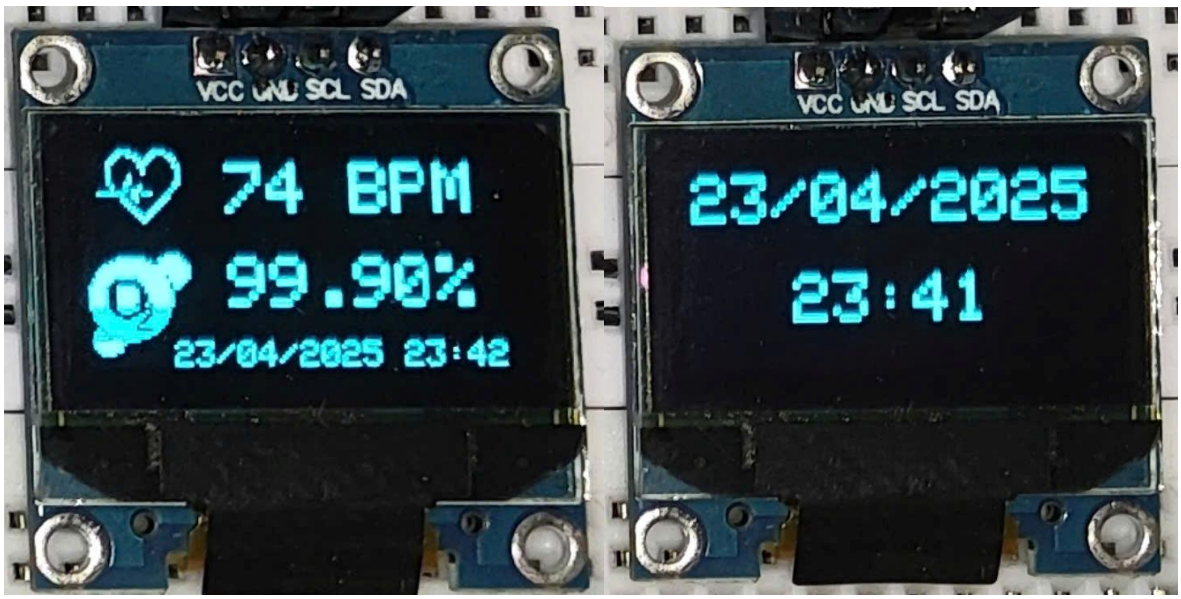
Hệ thống âm thanh và báo động khẩn:

+ *Thành tựu:* Nhóm lựa chọn âm thanh phù hợp dễ nghe không ảnh hưởng người xung quanh quá nhiều nhưng đảm bảo được người thân nhận ra tình trạng bất thường. Thông qua hai nền tảng Blynk và còi buzzer trên sản phẩm.

+ *Khó khăn:* Kết nối WiFi có thể bị gián đoạn, dẫn đến việc không gửi được dữ liệu hoặc thông báo qua Blynk. Blynk đôi khi không phản hồi kịp thời, ảnh hưởng đến việc gửi cảnh báo thời gian thực.

+ *Giải pháp:* Duy trì kết nối WiFi và Blynk. Sử dụng cơ chế dự phòng (Notify.cx, xDroid) khi Blynk không khả dụng, đáp ứng yêu cầu độ bền

Giao diện người dùng trực quan được minh họa:



Hình 4.12: Cấu hình màn OLED

+ *Thành tựu*: Màn hình OLED SSD1306 hiển thị nhịp tim, SpO2, thời gian và trạng thái hệ thống (WiFi, té ngã, khẩn cấp) với các biểu tượng (nhịp tim, oxy). Buzzer cung cấp cảnh báo âm thanh cục bộ, hữu ích trong môi trường không có kết nối.

+ *Khó khăn*: Còn quá đơn giản, chưa tận dụng không gian trống khi thông báo. Kích thước màn hình nhỏ, tiêu thụ bộ nhớ lớn và khả năng xử lý khi gặp ánh sáng mạnh cùng đó tần suất cập nhật màn hình.

+ *Giải pháp*: Cấu hình nội dung chi tiết tránh lãng phí tài nguyên này. Giảm khả năng tiêu thụ bộ nhớ và xử lý vấn đề khi gặp phải ánh sáng mạnh. Ngoài ra còn có thể tính toán để giảm thiểu khả năng tiêu tốn điện năng cao.

PHẦN 3: KẾT LUẬN

Hệ thống cảnh báo y tế dựa trên IoT cho người cao tuổi - ***Iot-based Medical Alert System For Elderly People*** đã đạt được những kết quả ấn tượng, mang lại giá trị thiết thực trong việc hỗ trợ chăm sóc sức khỏe và đảm bảo an toàn cho người cao tuổi. Với sự kết hợp của vi điều khiển ESP32 và các cảm biến hiện đại như MAX30102, MPU6050, GPS Neo-6M, hệ thống có khả năng giám sát nhịp tim, nồng độ oxy trong máu (SpO2), và phát hiện té ngã một cách tự động, đồng thời gửi cảnh báo kèm vị trí chính xác qua Blynk, Flask Server hoặc các dịch vụ dự phòng. Giao diện trực quan trên màn hình OLED SSD1306 cùng với khả năng giám sát từ xa thông qua Blynk và giao diện web đã giúp người thân và nhân viên y tế dễ dàng theo dõi tình trạng sức khỏe, kịp thời ứng phó với các tình huống khẩn cấp, từ đó mang lại sự an tâm và nâng cao chất lượng cuộc sống cho người cao tuổi.

Tuy nhiên, hệ thống vẫn tồn tại một số ưu và nhược điểm. Về ưu điểm, hệ thống nổi bật với tính năng thời gian thực, độ tin cậy cao nhờ cơ chế dự phòng thông báo và khả năng tích hợp đa nền tảng, đáp ứng tốt các yêu cầu về giao diện người dùng. Bên cạnh đó, việc sử dụng thuật toán 3 giai đoạn để phát hiện té ngã và quản lý trạng thái hiệu quả đã thể hiện sự tối ưu trong điều kiện tài nguyên hạn chế. Dẫu vậy, nhược điểm của hệ thống nằm ở độ chính xác của phát hiện té ngã, vốn phụ thuộc vào ngưỡng cố định và dễ bị ảnh hưởng bởi nhiễu từ cảm biến MPU6050. Ngoài ra, kết nối WiFi không ổn định trong một số trường hợp, cùng với hạn chế về thời lượng pin và bảo mật (sử dụng setInsecure()), là những vấn đề cần được khắc phục để đảm bảo hệ thống hoạt động bền vững trong môi trường thực tế.

Hướng phát triển trong tương lai của đề tài hứa hẹn sẽ mở ra nhiều tiềm năng để nâng cao hiệu quả và tính ứng dụng. Trước hết, việc tích hợp các thuật toán học máy nhẹ (như hồi quy hoặc cây quyết định) có thể cải thiện độ chính xác của phát hiện té ngã, đồng thời cho phép hệ thống thích nghi với các tình huống đa dạng hơn, đáp ứng tiêu chí đánh giá thuật toán. Bên cạnh đó, việc bổ sung module GSM để hỗ trợ gọi điện hoặc gửi SMS trực tiếp, cùng với chế độ ngủ (deep sleep) để tiết kiệm năng lượng, sẽ giúp hệ thống hoạt động lâu dài và đáng tin cậy hơn trong các điều kiện khắc nghiệt. Cuối cùng, cải thiện giao diện người dùng bằng cách tích hợp loa phát hướng dẫn bằng giọng nói và thiết kế thiết bị đeo tay nhỏ gọn sẽ tăng tính thân thiện, mở rộng khả năng ứng dụng không chỉ cho người cao tuổi mà còn cho các nhóm đối tượng khác như bệnh nhân hoặc người khuyết tật, góp phần xây dựng một xã hội an toàn và nhân văn hơn.

TÀI LIỆU THAM KHẢO

GIÁO TRÌNH

1. Phan Quốc Phô (2013), *Giáo trình Cảm biến Quyển 1*, nhà xuất bản Bách Khoa Hà Nội.
2. Phan Quốc Phô (2013), *Giáo trình Cảm biến Quyển 2*, nhà xuất bản Bách Khoa Hà Nội.

BÀI BÁO NGHIÊN CỨU

1. Šofranac Ivan, Ljucović Nikola,Radulović Ivana, Prof. dr Radovan Stojanović (2023), *Fall detection system for the elderly or disabled people,* Technical Report.
2. Nowshad Hasan, Mohammad Faisal Ahmed (2024), *Wearable Technology for Elderly Care: Integrating Health Monitoring and Emergency Alerts,* Journal of Computer Networks and Communications

NGUỒN INTERNET VÀ KHÁC

1. *IoT Fall detection using MPU6050 NodeMCU ESP8266 and Blynk App*. Đường dẫn tham khảo: https://www.youtube.com/watch?v=2oi-3aRuTFE&t=92s vào lúc 19h34 ngày 4/4/2025.
2. *IoT Fall Detector Using MPU6050 & ESP8266*. Đường dẫn tham khảo: https://iotprojectsideas.com/iot-fall-detector-using-mpu6050-esp8266/ vào lúc 12h08 ngày 5/4/2025.
3. *ESP32-MAX30102*. Đường dẫn tham khảo: https://github.com/youjunjer/ESP32-MAX30102/ vào lúc 14h27 ngày 5/4/2025.
4. *ESP32 with NEO-6M GPS Module (Arduino IDE)*. Đường dẫn tham khảo: https://randomnerdtutorials.com/esp32-neo-6m-gps-module-arduino/ vào lúc 20h39 ngày 6/4/2025.
5. *Az-Delivery for ESP32 Dev Kit C V2*. Đường dẫn tham khảo cho nội dung là https://www.edu.xunta.gal/centros/ieslaxeiro/system/files/ESP-32%20Dev%20Kit%20C%20V2_EN.pdf vào lúc 1h34 ngày 7/4/2025.
6. *Blynk.Apps*. Đường dẫn tham khảo https://docs.blynk.io/en/blynk.apps/overview vào lúc 2h19 ngày 7/4/2025.
7. *Components of the Blynk IoT Platform*. Đường dẫn tham khảo https://docs.blynk.io/en vào lúc 3h00 ngày 7/4/2025.

8. *Hướng dẫn sử dụng cảm biến gia tốc MPU6050 với Arduino*. Đường dẫn tham khảo: https://arduinokit.vn/huong-dan-su-dung-cam-bien-gia-toc-mpu6050-voi-arduino/ vào lúc 8h45 ngày 7/4/2025.
9. *Hướng dẫn sử dụng cảm biến nhịp tim và Oxy trong máu MAX30102 với Arduino.* Đường dẫn tham khảo: https://arduinokit.vn/huong-dan-su-dung-cam-bien-nhip-tim-va-oxy-trong-mau-max30102-arduino/ vào lúc 9h56 ngày 7/4/2025.
10. *Hướng dẫn sử dụng màn hình OLED 0.96 với Arduino (Phần 1).* Đường dẫn tham khảo https://arduinokit.vn/huong-dan-su-dung-man-hinh-oled-0-96-voi-arduino-phan-1/ vào lúc 11h34 ngày 9/4/2025.
11. *Mô-đun gia tốc kế MPU6050 và con quay hồi chuyển*. Đường dẫn tham khảo: https://mecsu.vn/ho-tro-ky-thuat/modun-gia-toc-ke-mpu6050-va-con-quay-hoi-chuyen.9Jx#:~:text=M%C3%B4-%C4%91un%20MPU6050%20l%C3%A0%20H%E1%BB%87%20th%E1%BB%91ng%20c%C6%A1%20%C4%91i%E1%BB%87n%20vi,kh%C3%A1c%20c%E1%BB%A7a%20m%E1%BB%99t%20h%E1%BB%87%20th%E1%BB%91ng%20ho%E1%BA%B7c%20v%E1%BA%ADt%20th%E1%BB%83 vào lúc 12h46 ngày 12/4/2025.
12. *Lập trình STM32 với Oled LCD SSD1306*. Đường dẫn tham khảo: https://khuenguyencreator.com/lap-trinh-stm32-voi-oled-lcd-ssd1306/ vào lúc 13h09 ngày 12/4/2025.
13. *Tìm hiểu về Buzzer*. Đường dẫn tham khảo: https://dientutuonglai.com/tim-hieu-buzzer.html vào lúc 14h45 ngày 12/4/2025.
14. *GPS NEO giao tiếp Arduino, định vị GPS Neo 6M + LCD1602 + Arduino*. Đường dẫn tham khảo: https://huynhnhattung.com/gps-neo-giao-tiep-arduino-dinh-vi-gps-neo-6m-lcd1602-arduino/ vào lúc 8h45 ngày 13/5/2025.